

Projet Speed Dating — Fiche de Révision

Certification CSDS — Formation Fullstack Data Engineering — Jedha

Aïcha FATHALLAH

1. Description générale du projet

Le projet Speed Dating consiste à analyser un dataset de **8 378 rencontres** issues d'expériences de speed dating organisées par Columbia University entre 2002 et 2004. L'objectif est d'aider l'équipe marketing de Tinder à comprendre **ce qui déclenche l'attraction** entre deux personnes, afin d'améliorer l'algorithme de matching.

Contexte métier

Tinder observe une baisse du nombre de matches. L'équipe marketing veut identifier les leviers objectifs de l'attraction — pas ce que les gens *croient* valoriser, mais ce qui *influence réellement* leur décision. Le dataset Columbia est idéal : chaque rencontre de 4 minutes a été suivie d'une décision secrète, ce qui permet de comparer déclarations et comportements réels.

Objectif final

- Répondre aux **5 questions clés** posées par le sujet avec des données chiffrées
- Produire des **statistiques descriptives** et des **visualisations** commentées
- Formuler des **recommandations stratégiques** actionnables pour Tinder

Architecture de l'analyse

Étape	Contenu	Outil utilisé
Chargement	Lecture du CSV 8 378 lignes × 195 colonnes	pandas read_csv
Nettoyage	Sélection 16 colonnes utiles, dropna, label genre	pandas DataFrame
Stats descriptives	describe(), biais de notation (std=0)	pandas + display()
Q1 — Genre	Préférences déclarées par genre (pf_o_*)	pandas groupby + bar chart
Q2 — Biais déclaratif	Importance déclarée vs. corrélation réelle	pandas corr() + barplot
Q3 — Intérêts vs race	Taux de match par samerace et par quartile int_corr	pandas + barplot
Q4 — Auto-perception	attr3_1 vs attr_o — biais d'auto-complaisance	pandas + barplot
Q5 — Ordre soirée	Taux de dec_o par tiers de la soirée + tendance	seaborn Implot + lineplot
Corrélations	Heatmap critères / dec_o, matrice globale, radar chart	seaborn + matplotlib polar
Conclusion	Tableau synthèse 5 questions + recommandations Tinder	Markdown structuré

2. Explication détaillée de chaque étape

Étape 1 — Chargement et exploration initiale

Le fichier Speed_Dating_Data.csv est chargé avec encoding='ISO-8859-1' (Latin-1) car le dataset contient des caractères spéciaux anglo-saxons. Sans ce paramètre, pandas lève une erreur `UnicodeDecodeError`. Le dataset brut

compte **8 378 lignes et 195 colonnes** — la plupart inutiles pour notre analyse.

Étape 2 — Nettoyage des données (Data Cleaning)

Le dataset brut de 195 colonnes est réduit à **16 colonnes essentielles** réparties en deux groupes :

- **score_cols** : les 6 notes reçues par le partenaire (suffixe `_o` = 'other') — `attr_o`, `sinc_o`, `intel_o`, `fun_o`, `amb_o`, `shar_o`
- **context_cols** : `iid`, `gender`, `match`, `dec_o`, `int_corr`, `order`, `age`, `samerace`

dropna() supprime les lignes incomplètes sur ces colonnes : on passe de 8 378 à **6 875 lignes** utilisables — quantité suffisante pour des analyses fiables. On ajoute un `.copy()` après la sélection pour éviter le *SettingWithCopyWarning* de pandas lors de la transformation suivante.

Transformation du genre : la colonne `gender` (0/1) est convertie en libellés explicites 'Femme'/'Homme' via une `lambda`, pour une meilleure lisibilité des graphiques.

Étape 3 — Statistiques descriptives

describe() sur les 6 critères de notation donne : `count`, `mean`, `std`, `min`, `quartiles`, `max`. Cela permet de vérifier les plages de valeurs et de détecter des anomalies.

Analyse des biais de notation : on groupe par `iid` et on calcule l'écart-type des notes d'attractivité données. Un `std == 0` signifie que le participant attribue toujours la même note à tout le monde — c'est un *biais de réponse*. Ces profils biaisent les moyennes du groupe mais leur proportion est marginale.

Étape 4 — Q1 : Attributs les moins désirables selon le genre

Les colonnes `pf_o_*` (partner's preferences about the opposite sex) contiennent ce que chaque participant *pense* que le sexe opposé valorise, réparti sur 100 points (somme = 100). On groupe par `gender` et on calcule la moyenne de chaque attribut.

Attribut	Femmes pensent que les hommes valorisent	Hommes pensent que les femmes valorisent
Attractivité	26.9 pts ← critère n°1	18.1 pts
Intelligence	19.5 pts	21.0 pts ← critère n°1
Sincérité	16.5 pts	18.3 pts
Humour	17.8 pts	17.1 pts
Ambition	8.6 pts ← le moins valorisé	12.8 pts ← le moins valorisé
Intérêts	11.0 pts	12.7 pts

Étape 5 — Q2 : Importance déclarée vs. impact réel de l'attractivité

On compare deux sources : **attr1_1** (importance que le participant dit accorder à l'attractivité, sur 100) et la **corrélation de `attr_o` avec `dec_o`** (impact réel sur la décision). On multiplie la corrélation par 100 pour rendre les deux échelles comparables visuellement dans le barplot.

Critère	Importance déclarée (/100)	Impact réel (corr. × 100)	Verdict
Attractivité	22.5	48.8	Sous-déclarée — biais !
Humour	17.5	41.2	Sous-déclaré
Intérêts	11.8	39.8	Très sous-déclaré
Intelligence	20.3	21.4	Honnête

Sincérité	17.4	20.8	Légèrement sur-déclarée
Ambition	10.7	18.3	Sous-déclarée

Étape 6 — Q3 : Intérêts communs vs. même origine raciale

On compare le taux de match selon deux variables : **samerace** (0/1) et **int_corr** découpée en quartiles (pd.qcut). La variable **samerace** est binaire : on compare directement les deux groupes. Pour **int_corr**, les quartiles permettent de voir si l'effet est progressif (plus d'intérêts communs = plus de matches).

Facteur	Groupe bas	Groupe haut	Écart
Même origine raciale	16.1% (origines diff.)	17.1% (même origine)	+1.0 pt
Intérêts communs	15.3% (Q1 — peu commun)	18.6% (Q4 — très commun)	+3.3 pts

Conclusion : les intérêts communs ont un impact 3x supérieur à l'origine raciale. On confirme avec la régression logistique (Implot, logistic=True) : la courbe est ascendante, ce qui valide l'effet progressif de **int_corr** sur la probabilité de match.

Étape 7 — Q4 : Les gens peuvent-ils prédire leur valeur perçue ?

On compare **attr3_1** (comment le participant pense être perçu) avec **attr_o** (la note réellement reçue de ses partenaires). On fait de même pour **sinc3_1** vs **sinc_o** et **intel3_1** vs **intel_o**. On crée un DataFrame dédié avec **.dropna()** sur ces 6 colonnes (7 914 lignes disponibles).

Attribut	Auto-perception (attr3_1)	Note réelle reçue (attr_o)	Surestimation
Attractivité	7.08 / 10	6.18 / 10	+0.90 pt
Sincérité	8.30 / 10	7.18 / 10	+1.13 pt
Intelligence	8.41 / 10	7.36 / 10	+1.05 pt

La **corrélation entre auto-perception et note réelle** = **0.175** seulement — très faible. Les participants surestiment systématiquement toutes leurs qualités. Ce phénomène s'appelle le **biais d'auto-complaisance** (self-serving bias).

Étape 8 — Q5 : Premier ou dernier rendez-vous de la soirée ?

La variable **order** indique la position du rendez-vous dans la soirée (1 = premier rencontré, max ≈ 22 = dernier). On découpe la soirée en 3 tiers avec **pd.cut()** et on calcule le taux moyen de **dec_o** (décision positive) pour chaque phase. On ajoute une ligne de tendance via **np.polyfit()** degré 1.

Phase de la soirée	Taux moyen de 'Oui' (dec_o)	Interprétation
Début (1er tiers)	43.2 %	Meilleure réceptivité
Milieu (2e tiers)	43.3 %	Légère baisse
Fin (3e tiers)	39.6 %	Fatigue décisionnelle

Résultat : 43.2 % pour les 3 premiers vs 39.6 % pour les 3 derniers = -3.6 pts. Mieux vaut être rencontré en début de soirée. La *fatigue décisionnelle* (decision fatigue) explique cette dégradation progressive de la qualité des évaluations.

Étape 9 — Corrélations et visualisations complémentaires

Visualisation	Code utilisé	Ce qu'on apprend
Countplot match	sns.countplot(hue='match')	Match rare ≈ 16% — données déséquilibrées

KDE âge / match	sns.kdeplot(fill=True)	L'âge n'influence pas le match
Boxplot attr_o	sns.boxplot(x='match', y='attr_o')	Médiane matchés >> non-matchés
Heatmap corr. / dec_o	df.corr()[['dec_o']]	Attractivité n°1, Ambition dernière
Matrice corrélation	sns.heatmap(corr_globale)	Effet de halo, lien Fun/Partage
Radar chart genre	matplotlib polar + fill()	Femmes > Hommes sur tous critères
Régression logistique	sns.lmplot(logistic=True)	Intérêts communs → probabilité match
Lineplot + tendance	np.polyfit + poly1d	Déclin progressif au fil de la soirée

3. Outils utilisés — à quoi ça sert et pourquoi ce choix

Outil / Bibliothèque	Rôle dans le projet	Pourquoi ce choix
pandas	Chargement CSV, nettoyage, groupby, merge	Standard industrie pour la manipulation de données tabulaires
numpy	Calcul des angles du radar chart (linspace, cos, sin)	Calcul numérique rapide et efficace
seaborn	countplot, kdeplot, boxplot, barplot, heatmap	Applique tout niveau sur matplotlib — moins de code pour des graphiques plus beaux
matplotlib	Radar chart polar, figure/axes, show()	Bibliothèque de base de la visualisation Python — seaborn en dépend
pd.qcut()	Découper int_corr en 4 quartiles d'égale fréquence	garantit un nombre équilibré d'observations par groupe
pd.cut()	Découper 'order' en 3 tiers de la soirée	cut est adapté ici car on veut des intervalles de valeurs fixes
corr()	Matrice de corrélation de Pearson entre les variables linéaires	Méthode linéaire standard pour quantifier l'association entre de
sns.lmplot(logistic=True)	Régression logistique intérêts → match	Match est binaire (0/1) → régression logistique adaptée vs ré
np.polyfit(deg=1)	Ligne de tendance sur le lineplot de la fatigue	Régression linéaire simple pour visualiser la direction général
%matplotlib inline	Affichage des graphiques dans le notebook	Magic command Jupyter — sans lui les graphiques s'ouvrent
dropna()	Suppression des lignes avec valeurs manquantes	Choix pragmatique : 8 378 → 6 875 lignes, largement suffisant
.copy()	Copie explicite du DataFrame après slicing	Évite le SettingWithCopyWarning de pandas lors des modifica

4. Questions probables du jury et réponses

Sur les données et les choix de nettoyage

Q : Pourquoi tu as choisi dropna() plutôt que d'imputer les valeurs manquantes ?

R : Parce que le dataset est suffisamment grand : on passe de 8 378 à 6 875 lignes, soit une perte de seulement 18 %. L'imputation (remplacer les NaN par la moyenne) aurait introduit un biais artificiel dans les corrélations — particulièrement problématique pour les analyses de Q2 et Q4 qui comparent précisément des valeurs réelles. Dans ce contexte exploratoire, la suppression est plus honnête.

Q : Pourquoi tu as sélectionné seulement 16 colonnes sur 195 ?

R : Les 195 colonnes incluent des questionnaires remplis à différents moments (avant, pendant, après les événements), des versions dupliquées des mêmes variables, et des données inutiles pour nos 5 questions. On sélectionne uniquement les colonnes directement liées à nos hypothèses : les 6 notes reçues (_o), les décisions (match, dec_o), et les variables contextuelles (genre, âge, order, samerace, int_corr).

Q : Qu'est-ce que le suffixe _o dans attr_o, sinc_o, etc. ?

R : _o signifie 'other' — c'est la note que l'autre personne (le partenaire) a attribuée au participant. Sans le suffixe (attr) c'est la note que le participant a attribué à l'autre. On utilise _o car on analyse ce que les gens reçoivent comme évaluation, pas ce qu'ils donnent.

Q : Pourquoi tu utilises .copy() après la sélection du DataFrame ?

R : Quand on sélectionne un sous-ensemble d'un DataFrame avec df[colonnes], pandas peut renvoyer une vue (view) plutôt qu'une copie. Si on modifie ensuite ce sous-ensemble (comme transformer la colonne gender), pandas lève un SettingWithCopyWarning car il ne sait pas si on veut modifier l'original ou la copie. Le .copy() force la création d'une copie indépendante et supprime l'ambiguïté.

Sur les visualisations

Q : Pourquoi tu utilises un KDE plot pour l'âge plutôt qu'un histogramme ?

R : Le KDE (Kernel Density Estimate) lisse la distribution et permet de superposer deux groupes (matchés / non-matchés) de façon lisible avec `fill=True`. Un histogramme avec deux groupes superposés serait difficile à lire. Le KDE révèle rapidement si les distributions se chevauchent — et ici elles se superposent quasi parfaitement, ce qui confirme que l'âge n'influence pas le match.

Q : Pourquoi un boxplot pour l'attractivité et pas un barplot ?

R : Le boxplot montre toute la distribution (médiane, quartiles, outliers) et non juste la moyenne. Ici c'est crucial : on veut montrer que les personnes matchées ont collectivement des scores plus élevés, pas juste en moyenne. La médiane du groupe matchés (≈ 7.8) vs non-matchés (≈ 5.6) est un argument bien plus fort qu'une simple différence de moyennes.

Q : Pourquoi multiplier la corrélation par 100 dans le graphique Q2 ?

R : Pour rendre les deux échelles comparables visuellement. L'importance déclarée est sur 100 (ex : 22.5), tandis que la corrélation est entre 0 et 1 (ex : 0.488). En multipliant par 100, on obtient 48.8 — un chiffre qu'on peut mettre côte à côte avec 22.5 dans le même graphique. C'est un choix de présentation, pas une transformation statistique.

Q : Pourquoi utiliser `pd.qcut()` pour `int_corr` et `pd.cut()` pour `order` ?

R : `qcut` découpe en quartiles d'égale fréquence : chaque groupe contient le même nombre d'observations. C'est adapté pour `int_corr` car la distribution n'est pas forcément uniforme — on veut des groupes équilibrés pour comparer les taux de match. `cut` en revanche découpe en intervalles de valeurs fixes, adapté pour `order` où on veut délimiter des phases temporelles réelles (0-33% de la soirée, 33-66%, 66-100%).

Q : Qu'est-ce que la régression logistique et pourquoi l'utiliser ici ?

R : La régression logistique modélise la probabilité d'un événement binaire (0 ou 1) en fonction d'une variable continue. Ici, `match` est binaire et `int_corr` est continue : on veut savoir si plus d'intérêts communs augmente la probabilité de match. Une régression linéaire serait incorrecte car elle pourrait prédire des probabilités négatives ou supérieures à 1. `logistic=True` dans `sns.lmplot()` applique automatiquement la bonne transformation.

Q : Comment tu interprètes l'effet de halo dans la matrice de corrélation ?

R : L'effet de halo est un biais cognitif : quand on juge quelqu'un très attirant physiquement, on a tendance à lui attribuer inconsciemment de meilleures notes sur tous les autres critères (intelligence, humour...). Dans la matrice, on le voit concrètement : `attr_o` est corrélé positivement avec `fun_o` et `intel_o`. Cela signifie que les notes de personnalité ne sont pas indépendantes de la note physique — ce qui rend nos données partiellement subjectives.

Sur les résultats et les conclusions

Q : Les intérêts communs ont un impact 3× plus fort que l'origine raciale — comment tu le calcules ?

R : L'écart lié à l'origine raciale est de +1.0 pt (17.1% - 16.1%). L'écart lié aux intérêts communs est de +3.3 pts (18.6% - 15.3%). $3.3 / 1.0 \approx 3.3$, soit environ 3 fois plus d'impact. Ce n'est pas une corrélation mais une comparaison de taux de match entre groupes extrêmes (Q1 vs Q4, `samerace 0` vs 1).

Q : Comment tu expliques que le taux de match soit identique pour les hommes et les femmes ?

R : `Match = 1` seulement si les DEUX personnes ont dit Oui. C'est donc symétrique par construction : si X hommes ont matché, exactement X femmes ont aussi matché. Le taux identique ($\approx 16.5\%$) n'est pas surprenant — ce qu'il révèle, c'est que les deux genres sont également sélectifs, même si leurs critères diffèrent (Q1).

Q : La fatigue décisionnelle de -3.6 pts est-elle statistiquement significative ?

R : Dans notre analyse exploratoire, on ne réalise pas de test statistique formel (t-test, ANOVA). On observe une tendance visible et cohérente avec la littérature psychologique sur la fatigue décisionnelle. Pour valider statistiquement ce résultat, il faudrait un test de différence de moyennes entre les deux groupes. C'est une limite que je peux reconnaître si le jury la soulève.

Q : Pourquoi les gens surestiment-ils systématiquement leurs qualités ?

R : C'est le biais d'auto-complaisance (self-serving bias) : un mécanisme psychologique qui protège l'estime de soi. On a naturellement tendance à s'attribuer plus de qualités qu'on n'en a réellement. La corrélation de 0.175 entre auto-perception et note réelle montre que les gens ont une vision très floue de leur attractivité objective — ce qui

explique en partie les frustrations sur les applications de dating.

Questions pièges

Q : Ton analyse prouve-t-elle la causalité ou juste la corrélation ?

R : Seulement la corrélation. On ne peut pas affirmer que l'attractivité CAUSE le match — peut-être que d'autres variables cachées (personnalité, contexte, chimie) expliquent les deux simultanément. Pour établir la causalité, il faudrait une expérience contrôlée randomisée (RCT). Notre analyse est exploratoire et descriptive, ce qui est exactement ce qui est attendu pour ce projet.

Q : Le dataset de 2002-2004 est-il encore pertinent pour Tinder aujourd'hui ?

R : Les biais cognitifs fondamentaux (effet de halo, biais d'auto-complaisance, fatigue décisionnelle) sont des phénomènes psychologiques stables dans le temps. Cependant, les préférences culturelles peuvent avoir évolué en 20 ans — notamment sur l'importance accordée à l'ambition ou aux intérêts partagés. Ce dataset reste académiquement valide pour une analyse exploratoire, mais une validation sur des données Tinder actuelles serait idéale en production.

Q : Pourquoi tu n'as pas fait de machine learning pour prédire le match ?

R : Le sujet demande explicitement de l'EDA (Exploratory Data Analysis) : statistiques descriptives et visualisations. Le machine learning (classification binaire) serait une étape suivante naturelle — par exemple un modèle Random Forest pour identifier les features les plus importantes. Mais ce n'était pas dans le scope du projet. Je peux l'évoquer comme perspective d'amélioration.

Q : Qu'est-ce que tu améliorerais si tu avais plus de temps ?

R : Plusieurs axes : (1) Ajouter des tests statistiques formels (t-test, chi-2) pour valider les différences observées. (2) Analyser les données longitudinales du dataset — les questionnaires remplis avant et après l'événement permettent de voir l'évolution des perceptions. (3) Construire un modèle prédictif (Random Forest, Logistic Regression) avec validation croisée pour identifier les variables les plus prédictives du match. (4) Corriger le biais d'imbalanced data (84% de non-matches) avec SMOTE ou class_weight.

5. Synthèse — Réponses aux 5 questions du sujet

Question du sujet	Réponse chiffrée	Visualisation utilisée
Q1 — Attribut le moins désirable pour un partenaire masculin / féminin ?	Ambition — 8.6/100 pour les femmes, 12.8/100 pour les hommes. Femmes pensent que les hommes veulent surtout le physique (26.9 pts). Hommes surestiment l'importance de l'intelligence pour les femmes.	Barplot groupé (pf_o_* par genre)
Q2 — L'attractivité est-elle sous/sur-estimée vs. son impact réel ?	Sous-estimée : déclarée 22.5/100 mais impact réel = 0.49 (n°1). Humour sous-déclaré (17.5 vs 0.41). Sincérité sur-déclarée vs impact réel. Biais de désirabilité sociale prouvé.	Double barplot importance déclarée vs impact réel
Q3 — Intérêts communs plus importants que même origine raciale ?	OUI. Même origine : +1.0 pt de match. Intérêts communs : +3.3 pts de match. Impact 3× supérieur. Régression logistique confirme l'effet progressif.	Barplot samerace + barplot quartiles + Implot logistique
Q4 — Les gens prédisent-ils bien leur valeur perçue dans le dating ?	NON. Surestimation sur tous les critères : +0.90 pt (attractivité), +1.13 pt (sincérité), +1.05 pt (intelligence). Corrélation auto-perc / réel = 0.175.	Double barplot attr3_1 vs attr_o (+ sinc, intel)
Q5 — 1er ou dernier rendez-vous de la soirée pour un 2e rendez-vous ?	1er rendez-vous. Taux de 'Oui' : 43.2 % en début de soirée vs 39.6 % en fin → -3.6 pts. Fatigue décisionnelle confirmée par la ligne de tendance décroissante.	Lineplot + tendance np.polyfit() + barplot 3 phases

6. Glossaire rapide

Terme	Définition
EDA	Exploratory Data Analysis : analyse exploratoire pour comprendre un dataset avant modélisation
DataFrame	Structure de données tabulaire de pandas (lignes × colonnes), équivalent d'un tableau Excel en Python
dropna()	Supprime les lignes contenant au moins une valeur manquante (NaN) dans les colonnes spécifiées
corr()	Calcule la matrice de corrélation de Pearson entre toutes les variables numériques d'un DataFrame
groupby()	Agrège le DataFrame par groupe (ex : par genre) pour calculer des statistiques par catégorie
pd.qcut()	Découpe une variable continue en N groupes d'égale fréquence (quartiles, déciles...)
pd.cut()	Découpe une variable continue en intervalles de valeurs fixes définis manuellement
KDE	Kernel Density Estimate : estimation de la densité de probabilité, version lissée d'un histogramme
Boxplot	Graphique montrant médiane, quartiles et outliers — plus informatif qu'une simple moyenne
Heatmap	Carte de chaleur colorée représentant l'intensité d'une valeur (ex : coefficient de corrélation)

Radar chart	Graphique en toile d'araignée pour comparer plusieurs dimensions simultanément entre groupes
Régression logistique	Modèle statistique adapté aux variables cibles binaires (0/1) — prédit une probabilité
Corrélation de Pearson	Mesure linéaire entre -1 et +1 de l'association entre deux variables continues
Biais de désirabilité sociale	Tendance à déclarer ce qu'on pense être valorisé socialement plutôt que la réalité
Biais d'auto-complaisance	Tendance à se percevoir plus positivement que les autres ne nous voient
Effet de halo	Biais cognitif : une qualité perçue (ex : beauté) biaise positivement l'évaluation des autres traits
Fatigue décisionnelle	Dégradation de la qualité des décisions après une série de choix répétés
Imbalanced data	Dataset déséquilibré où une classe est très majoritaire (ici 84% de non-matches)
dec_o	Variable binaire : décision de l'autre personne (1 = elle veut revoir le participant, 0 = non)
match	Variable binaire : 1 seulement si les DEUX personnes ont dit Oui — intersection de dec_o des deux c
pf_o_*	Colonnes de préférences déclarées : ce que le participant pense que le sexe opposé valorise (sur 100)
attr3_1	Auto-évaluation de l'attractivité : comment le participant pense être perçu par les autres
int_corr	Score de corrélation des intérêts entre les deux partenaires (calculé à partir de questionnaires)
samerace	Variable binaire : 1 si les deux partenaires appartiennent au même groupe racial
order	Numéro de passage du rendez-vous dans la soirée (1 = premier partenaire rencontré)